Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»

Факультет компьютерных наук
Департамент программной инженерии

ОТЧЁТ

о выполнении индивидуального задания №1
по дисциплине «Архитектура компьютера и операционные системы»

Разработка последовательной имитационной программы

Вариант 32. Кухня ресторана

Выполнил: студент группы БПИ255
Кирпаль Андрей Николаевич

Москва 2026

1. Условие задачи

Вариант 32. Кухня ресторана. В ресторан приходят посетители, формирующие заказы из одного или нескольких блюд. Официант принимает заказ и передаёт его на кухню. Каждое блюдо состоит из одной или нескольких операций, для которых требуются повар и определённое оборудование: плита, печь, гриль или рабочий стол.

Повара выбирают доступные операции из очереди. Одна единица оборудования одновременно используется не более чем для одной операции. Готовые блюда собираются по номеру заказа. Официант может передать заказ посетителю только после приготовления всех входящих в него блюд. Вместимость очередей кухни и зоны готовых блюд может быть ограничена. Требуется разработать консольное приложение, моделирующее работу ресторана.

Входные параметры: количество посетителей, официантов и поваров; перечень блюд и последовательности операций; количество единиц оборудования каждого типа; состав заказов; интервалы появления посетителей; время приготовления операций; вместимость промежуточных очередей; правила приоритета заказов.

Отображаемые события: приход посетителя, принятие заказа, создание операций, постановка в очередь, получение поваром оборудования, начало и окончание приготовления, готовность блюда, сборка и выдача заказа.

Инварианты: одну единицу оборудования использует не более одного повара; повар выполняет не более одной операции; операция начинается только после выполнения зависимых предыдущих операций; блюдо считается готовым после завершения всех его операций; заказ выдаётся только после готовности всех входящих блюд и только своему посетителю.

Завершение: после закрытия приёма заказов и обслуживания всех принятых посетителей, либо после явной отмены невыполнимых заказов, либо по прерыванию пользователем, если задан режим неограниченного по времени выполнения. Посетители, официанты, повара и координатор кухни являются самостоятельными участниками модели.

2. Сценарий и устройство программы

Программа написана на C, состоит из трёх файлов: `restaurant.h` (типы данных), `main.c` (параметры, файл настроек, аргументы, сигналы, вывод) и `sim.c` (модель). Программа последовательная — всё выполняется в одном потоке. Время модели идёт в минутах. В календаре событий хранятся приход следующего посетителя и моменты окончания текущих действий: официант принял заказ, официант донёс заказ, повар закончил операцию. Основной цикл (`run_simulation`) берёт ближайшее событие, переводит часы на его время, обрабатывает событие, а затем по очереди вызывает координатора кухни, поваров и официантов, пока кто-нибудь из них может сделать что-то новое. Так в одном потоке моделируется одновременная работа нескольких участников.

Таблица 1 — Участники модели и их представление в программе

Сущность	Тип данных	Функции
Посетитель и его заказ	`Order` (у посетителя один заказ, номер заказа = номер посетителя)	`visitor_arrives`
Официант	`Waiter`	`waiters_work`, `order_taken`, `order_delivered`
Координатор кухни	очереди `tray`, `cooking`, `pickup`, `queue`	`coordinator`, `impossible`
Повар	`Cook`	`cooks_work`, `op_cooked`
Оборудование	массив `unit_user` (какой повар занял единицу)	`free_unit`
Блюдо, операция	`Recipe`, `Step`, `Op`	—
Параметры	`Params`	`set_defaults`, `load_config`, `check_params`

Путь заказа через модель:

1. Посетитель приходит и сразу выбирает блюда (случайно, от `order_min` до `order_max`). Он встаёт в очередь зала `hall`; планируется приход следующего посетителя.
2. Свободный официант подходит к посетителю. Через `take_time` минут заказ принят и передан на кухню — в лоток `tray`.
3. Координатор отменяет невыполнимый заказ: если нет поваров, нет нужного оборудования или блюд больше, чем мест в зоне выдачи. Остальные заказы он берёт в работу, если в зоне выдачи хватает мест на все блюда, и бронирует эти места. Затем ставит следующую операцию каждого блюда в очередь кухни `queue`, пока в ней есть место.
4. Свободный повар берёт из очереди операцию, для которой свободно оборудование, занимает его и готовит. По окончании (событие) он освобождает оборудование.
5. Когда выполнена последняя операция блюда, блюдо готово и кладётся в зону выдачи. Когда готовы все блюда, заказ собран и попадает в список `pickup`.
6. Свободный официант в первую очередь забирает собранные заказы и несёт их посетителю. Посетитель получает заказ и уходит.

Места в зоне выдачи бронируются при приёме заказа в работу, потому что без этого зона могла бы заполниться частями разных заказов — тогда ни один заказ нельзя было бы собрать, и модель зависла бы.

Правило приоритета задаётся параметром `priority`: `fifo` — всё по порядку; `vip` — VIP-посетителей официанты обслуживают первыми, а операции их заказов первыми попадают в очередь и первыми берутся поварами.

Инварианты. Повар берёт только свободную единицу оборудования, при захвате это проверяется. У повара одно поле для операции, и работу берёт только свободный повар. Следующая операция блюда ставится в очередь только после окончания предыдущей (флаг `in_work`). Блюдо готово, когда выполнены все его операции. При выдаче проверяется, что готовы все блюда заказа; заказ хранится в одной структуре со своим посетителем, поэтому отдать его чужому невозможно. Проверки выполняет функция `check`: при нарушении выводится строка «НАРУШЕНИЕ», оно учитывается в итогах, код завершения — 3.

3. Пользовательский интерфейс

Параметры имеют значения по умолчанию в программе (`set_defaults`), могут задаваться в файле настроек (ключ `-c`) и ключами командной строки; ключи важнее файла. Файл настроек состоит из строк «параметр значение», строки с `#` — комментарии.

Таблица 2 — Параметры файла настроек

Параметр	Смысл	По умолчанию
visitors	число посетителей (0 — без ограничения)	15
waiters, cooks	число официантов и поваров	2, 4
stove, oven, grill, table	число плит, печей, грилей, рабочих столов	2, 1, 2, 2
dish	блюдо меню и его операции	5 блюд
order МИН МАКС	сколько блюд в заказе	1 3
arrive МИН МАКС	интервал между приходами посетителей, мин	1 5
take_time, deliver_time	время принятия и доставки заказа, мин	2, 1
queue_cap, pass_cap	вместимость очереди операций и зоны выдачи	8, 8
priority, vip_percent	правило приоритета (fifo/vip), доля VIP, %	fifo, 15
seed, delay, log	seed, задержка минуты в мс, файл журнала	время, 200, restaurant.log

Блюдо записывается как название и список операций «операция:оборудование:минуты», например: `dish Омлет взбить:table:1 жарка:stove:4`. Первое блюдо из файла заменяет меню по умолчанию.

Ключи командной строки: `-c файл` — файл настроек, `-n N` — число посетителей (0 — без ограничения), `-s N` — seed, `-d мс` — задержка показа одной модельной минуты, `-l файл` — журнал, `-h` — справка.

Программа выводит параметры, а затем события в виде «[время] Участник номер: событие» (пример — в разделе 5). Для наглядности между событиями выдерживается задержка (`nanosleep`). В конце печатаются итоги: причина завершения, сколько посетителей пришло, обслужено и отменено, среднее и наибольшее время в ресторане (отдельно для VIP), загрузка поваров, число проверок инвариантов.

Ввод-вывод через файловые дескрипторы. Файл настроек открывается `open` и читается `read`. Вывод выполняется функциями `dprintf`/`vdprintf` в дескриптор 1 и одновременно в дескриптор журнала, открытого `open(..., O_WRONLY | O_CREAT | O_TRUNC, 0644)`; ошибки — в дескриптор 2. Журнал содержит все события и итоги и закрывается `close` при завершении.

4. Варианты запуска и завершения

Таблица 3 — Варианты запуска и завершения

Запуск	Завершение	Код
`./restaurant`	пришли все 15 посетителей, все обслужены	0
`./restaurant -c tests/no_grill.cfg`	невыполнимые заказы явно отменены, остальные обслужены	0
`./restaurant -n 0`	посетители приходят бесконечно; по Ctrl+C — итоги и журнал	0
Ctrl+C при конечном числе посетителей	моделирование прервано, в итогах — сколько не обслужено	1
повторное Ctrl+C	немедленный выход	130
ошибка в файле или ключах	сообщение об ошибке	2
нарушение инварианта или тупик	сообщение и итоги	3

Ctrl+C обрабатывается через `sigaction`. Обработчик только запоминает номер сигнала в переменной типа `volatile sig_atomic_t`, а основной цикл проверяет её после каждого события, выводит итоги и сохраняет журнал. Флаг SA_RESTART не ставится, поэтому сигнал сразу прерывает паузу `nanosleep` и программа реагирует мгновенно.

5. Результаты работы

Запуск `./restaurant -s 42` (параметры по умолчанию). Фрагмент вывода — начало работы и путь заказа №1:

```
[12:00] Ресторан: открыт
[12:00] Посетитель 1: пришёл и ждёт официанта (в зале ждут: 1)
[12:00] Официант 1: подошёл к посетителю #1
[12:02] Посетитель 2: пришёл (VIP) и ждёт официанта (в зале ждут: 1)
[12:02] Официант 2: подошёл к посетителю #2
[12:02] Официант 1: принял заказ #1 (Блины, Цезарь) и передал его на кухню
[12:02] Координатор: заказ #1 принят в работу, создано операций: 4
[12:02] Координатор: «тесто» (стол) для блюда «Блины» заказа #1 → очередь [1/8]
[12:02] Координатор: «курица» (гриль) для блюда «Цезарь» заказа #1 → очередь [2/8]
[12:02] Повар 1: взял «тесто» для блюда «Блины» заказа #1 и получил оборудование: стол 1
[12:02] Повар 1: начал «тесто» (2 мин)
[12:02] Повар 2: взял «курица» для блюда «Цезарь» заказа #1 и получил оборудование: гриль 1
[12:02] Повар 2: начал «курица» (5 мин)
[12:04] Официант 2: принял заказ #2 (Стейк, Стейк, Стейк) и передал его на кухню
...
[12:11] Повар 3: закончил «салат» для блюда «Цезарь» заказа #1 и освободил: стол 1
[12:11] Координатор: блюдо «Цезарь» заказа #1 готово → зона выдачи [2/8]
[12:11] Координатор: заказ #1 собран (блюд: 2) и ждёт официанта
[12:11] Официант 2: забрал заказ #1 из зоны выдачи
[12:12] Официант 2: выдал заказ #1 посетителю #1
[12:12] Посетитель 1: получил заказ и ушёл (пробыл в ресторане 12 мин)
```

Итоги этого запуска:

```
=== Итоги ===
Завершение: приём закрыт, все посетители обслужены
Модельное время: 96 мин
Посетителей пришло: 15, обслужено: 15, заказов отменено: 0, не обслужено: 0
Время в ресторане: среднее 38 мин, наибольшее 61 мин
  VIP: 39 мин, остальные: 37 мин
Повар 1: операций 18, занят 96% времени
Повар 2: операций 23, занят 85% времени
Повар 3: операций 18, занят 83% времени
Повар 4: операций 18, занят 72% времени
Проверок инвариантов: 310, нарушений: 0
```

Все 15 посетителей обслужены, в среднем за 38 минут. Повара заняты 72–96 % времени, то есть узкое место ресторана — кухня. Нарушений инвариантов нет.

Если на кухне нет гриля (`tests/no_grill.cfg`), координатор отменяет заказы, где он нужен:

```
[12:02] Координатор: заказ #1 невыполним (для блюда «Стейк» нужно оборудование «гриль», а его нет) — отменён
```

6. Тестирование

Тесты запускаются командой `make test` (скрипт `tests/run_tests.sh`). Для каждого файла `tests/*.cfg` программа запускается без задержки; проверяются код завершения, наличие ожидаемых строк в выводе (они записаны в самом файле в строках `# expect:`) и отсутствие нарушений инвариантов.

Таблица 4 — Тесты

Тест	Что проверяется
basic.cfg	обычная работа, все посетители обслужены
small_kitchen.cfg	1 официант, 1 повар, по одной единице оборудования, очередь на 2 операции
no_grill.cfg	нет гриля — заказы со стейком и цезарем отменяются
small_pass.cfg	зона выдачи на 2 блюда — большие заказы отменяются, остальные ждут места
vip_menu.cfg	своё меню из файла и правило приоритета vip
bad.cfg	ошибка в файле настроек — код 2
Ctrl+C	режим без ограничения останавливается сигналом SIGINT, итоги сохраняются в журнале

Все 7 тестов проходят на macOS (clang) и на Ubuntu 24.04 (gcc); программа компилируется с ключами `-Wall -Wextra` без предупреждений.

7. Использование ИИ

7.1. Промпты

Следующие промпты запускались в режиме агента с возможностью редактировать файлы с кодом и запускать их. Задание и требования лежали в контексте.

1. «добавь комментарии к коду и тесты».
2. «теперь напиши отчёт в виде вордовского файла по заданию в соответствии с требованиями, раздел использование ии оставь незаполненным».

7.2. Критическая оценка программы и внесённые правки

1. Код был написан мною, работа ИИ касалась только тривиальных задач.
2. Ошибок в комментариях и тексте отчета не обнаружено.